

Spotify SQL Data Analysis

Insights derived from PostgreSQL and SQL

Data Analyst Portfolio Project

By- Sameer Raj



Contents

- Introduction & project overview
- Dataset and data preparation
- Objective and analytical methodology
- Exploratory data analysis
- SQL analysis: Questions 1–15
- SQL concepts demonstrated
- Insights, challenges and recommendations
- Conclusion

Introduction

Project overview

- Analyze track-level Spotify data using PostgreSQL.
- Explore artists, albums, engagement metrics, streaming behavior and audio characteristics.
- Translate analytical questions into progressively more advanced SQL queries.
- Use aggregation, conditional logic, subqueries, CTEs and window functions.

SQL rows
20,592
shown in
pgAdmin

Columns
24
dataset fields

Artists
2,074
distinct

Albums
11,854
distinct

Data Preparation

Dataset structure and scope

- The dataset contains 24 columns covering track metadata, audio features and engagement metrics.
- Core fields include Artist, Track, Album, Album_type, Views, Likes, Comments, Stream, Energy, Liveness, Licensed and official_video.
- The SQL table was explored in pgAdmin before analysis.
- Data types include text, numeric and boolean fields.

Artist	Track	Album	Album_type	Danceability	Energy
Loudness	Speechiness	Acousticness	Instrumentalness	Liveness	Valence
Tempo	Duration_min	Title	Channel	Views	Likes
Comments	Licensed	official_video	Stream	EnergyLiveness	most_playedon

Objective

- Understand the structure and distribution of the Spotify dataset.
- Identify high-performing tracks, albums and artists.
- Compare engagement metrics such as views, likes, comments and streams.
- Analyze audio attributes such as energy, danceability and liveness.
- Demonstrate SQL techniques that are directly useful in data analyst work.

Methodology

SQL techniques used

Filtering

WHERE, DISTINCT

Aggregation

COUNT, SUM, AVG, MIN, MAX

Grouping

GROUP BY, ORDER BY

Conditional logic

CASE WHEN, COALESCE

Advanced SQL

Subqueries, CTEs

Window functions

DENSE_RANK, SUM() OVER

Safe calculations

NULLIF for division

Output review

pgAdmin Data Output

Exploratory Data Analysis

Query

Query History

Scratch Pad

1

-- EDA-----

2

select * from spotify;

3

Data Output

Messages

Notifications

- SQL table: 20,592 rows and 24 columns.
- 2,074 distinct artists and 11,854 distinct albums.
- Album types: album, single, compilation.
- Most-played platforms in the data: Spotify, Youtube.
- Track duration ranges from roughly 0.52 to 77.93 minutes in the displayed SQL checks.

EDA Snapshot

Key distributions observed in the SQL outputs

Query	Query History	
5		
6	<code>select count(distinct artist) from spotify;</code>	
7		
Data Output	Messages	Notifications
         		
	count bigint 	
1	2074	

Query

Query History

7

8

9

select count(distinct album) from spotify;

Data Output

Messages

Notifications

SQL

count

bigint

1

11853

Query	Query History	
9		
10	<code>select distinct album_type from spotify;</code>	
11		
Data Output	Messages	Notifications
		
	album_type character varying (50) 🔒	
1	album	
2	compilation	
3	single	

Query	Query History	
16	<code>select distinct most_played_on from spotify;</code>	
17		
18	<code>select max(duration_min) from spotify;</code>	
Data Output	Messages	Notifications
		
	most_played_on character varying (50)	
1	Youtube	
2	Spotify	

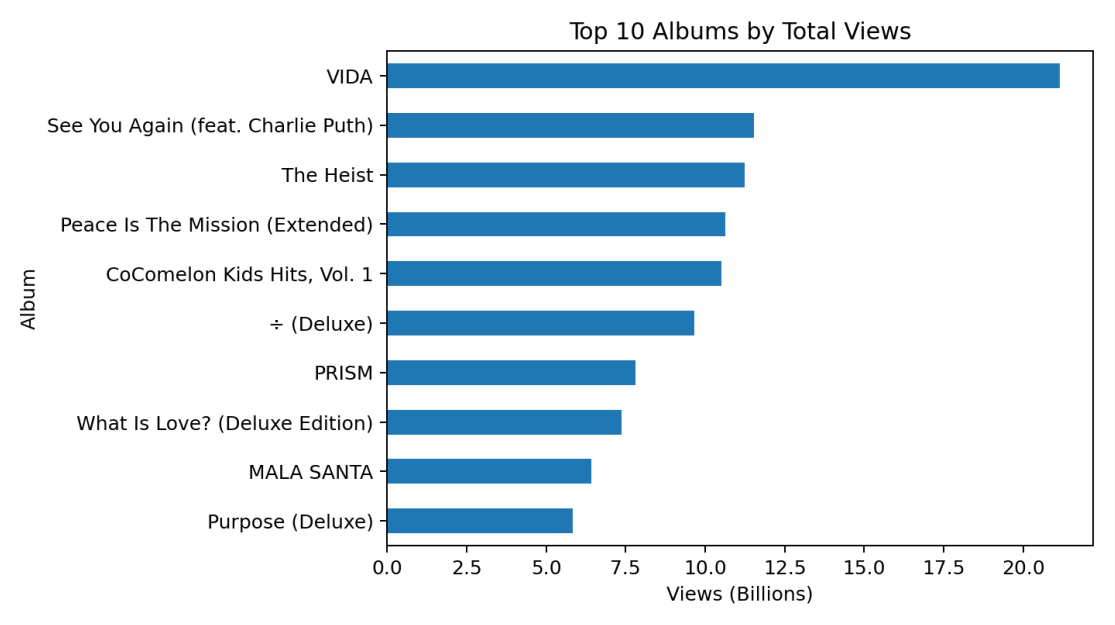
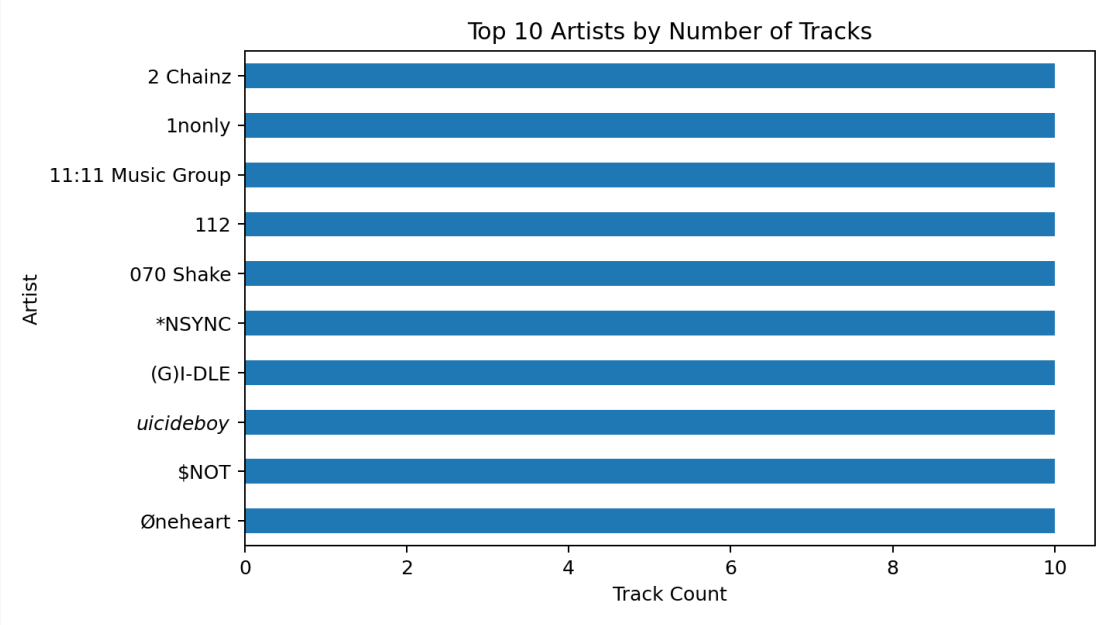
Query	Query History	
17		
18	<code>select max(duration_min) from spotify;</code>	
19	<code>select min(duration_min) from spotify;</code>	
...		
Data Output	Messages	Notifications
	max double precision	
1	77.9343	

Query	Query History	
17		
18	<code>select max(duration_min) from spotify;</code>	
19	<code>select min(duration_min) from spotify;</code>	
Data Output	Messages	Notifications
         		
	min double precision 	
1	0.516416667	

- The dataset has substantial variety in artists and albums.
- Boolean fields such as Licensed and official_video contain both TRUE and FALSE values.
- Duration checks help identify unusually short or long records before deeper analysis.

Visual Insights

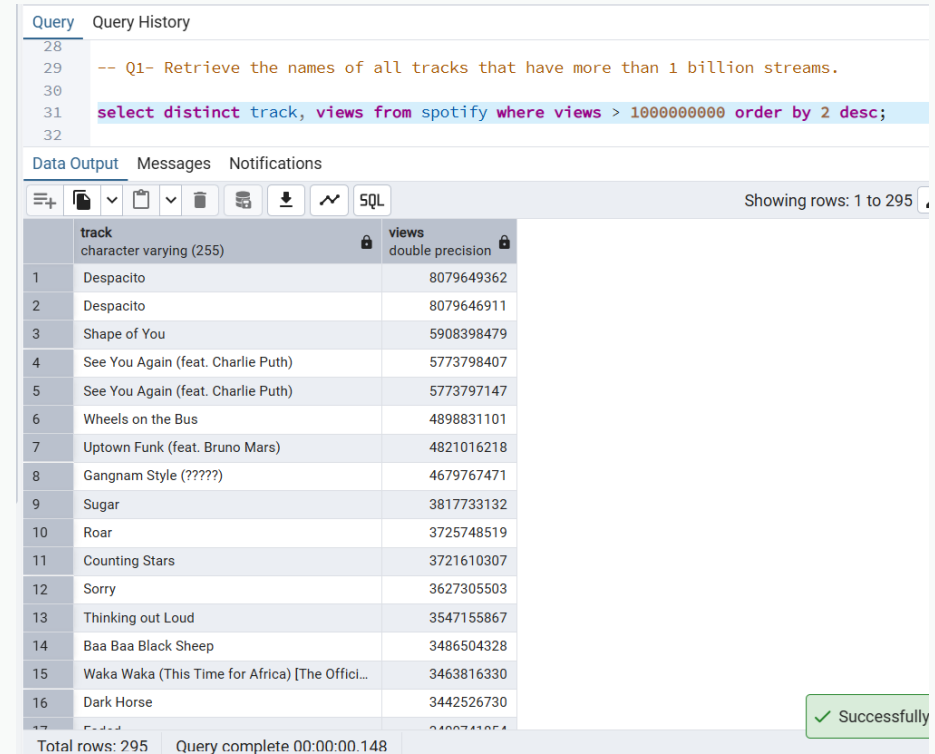
Derived from the dataset



Q1 — SQL Analysis

Retrieve the names of all tracks that have more than 1 billion streams.

```
SELECT DISTINCT track, views
FROM spotify
WHERE views > 1000000000
ORDER BY 2 DESC;
```



The screenshot shows a SQL query interface with a 'Query' tab and a 'Data Output' tab. The query is: `-- Q1- Retrieve the names of all tracks that have more than 1 billion streams.`
`select distinct track, views from spotify where views > 1000000000 order by 2 desc;`

The 'Data Output' tab shows a table with 295 rows. The table has two columns: 'track' (character varying (255)) and 'views' (double precision). The results are sorted by views in descending order.

	track	views
1	Despacito	8079649362
2	Despacito	8079646911
3	Shape of You	5908398479
4	See You Again (feat. Charlie Puth)	5773798407
5	See You Again (feat. Charlie Puth)	5773797147
6	Wheels on the Bus	4898831101
7	Uptown Funk (feat. Bruno Mars)	4821016218
8	Gangnam Style (?????)	4679767471
9	Sugar	3817733132
10	Roar	3725748519
11	Counting Stars	3721610307
12	Sorry	3627305503
13	Thinking out Loud	3547155867
14	Baa Baa Black Sheep	3486504328
15	Waka Waka (This Time for Africa) [The Offici...	3463816330
16	Dark Horse	3442526730
17	...	...

Total rows: 295 Query complete 00:00:00.148

Showing rows: 1 to 295

Successfully

INTERPRETATION:

The query filters high-performing tracks and returns 295 qualifying records in the screenshot. Sorting by views highlights the most-viewed tracks first.

Q2 — SQL Analysis

List all albums along with their respective artists.

```
SELECT DISTINCT album, artist
FROM spotify;
```

Query

Query History

```

32
33 -- Q2- List all albums along with their respective artists.
34
35 select distinct album, artist from spotify;
36

```

Data Output

Messages

Notifications

≡

📁

▼

📁

▼

🗑️

📦

⬇️

📶

SQL

	album character varying (255)	artist character varying (255)
1	NEED A FAVOR	Jelly Roll
2	Agar Tum Na Hote (Original Motion Picture So...	R. D. Burman
3	Algo De Mi	Conjunto Primavera
4	The Melodic Blue (Deluxe)	Baby Keem
5	A N N I V E R S A R Y	Bryson Tiller
6	Untouchables	Korn
7	Alright, Still (Bonus Track Version)	Lily Allen
8	Mozart: Requiem	Berliner Philharmoniker
9	#NAME?	Jesse & Joy
10	HUMBLE. (SKRILLEX REMIX)	Skrillex
11	Moondance (Deluxe Edition)	Van Morrison
12	Corridos Pesadísimos	Grupo Laberinto
13	Formula of Love: O+T=∞3	TWICE
14	Elvis 30 #1 Hits (Expanded Edition)	Elvis Presley
15	Baby Beluga	Raffi
16	Romanza (Remastered)	Andrea Bocelli
17	Everlasting	Silk Sonic

Total rows: 14178

Query complete 00:00:00.095

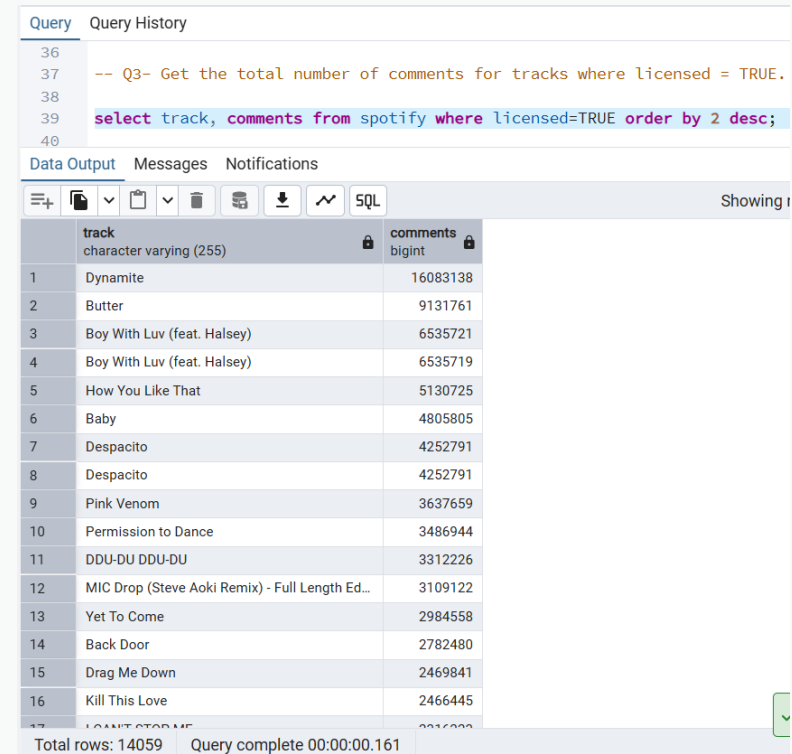
INTERPRETATION:

DISTINCT removes duplicate album–artist combinations and produces a clean mapping between albums and artists.

Q3 — SQL Analysis

List tracks and their comments where licensed = TRUE.

```
SELECT track, comments
FROM spotify
WHERE licensed = TRUE
ORDER BY 2 DESC;
```



The screenshot shows a SQL query interface with a 'Query' tab and a 'Data Output' tab. The query is: `-- Q3- Get the total number of comments for tracks where licensed = TRUE.`
`select track, comments from spotify where licensed=TRUE order by 2 desc;`

The 'Data Output' tab shows a table with two columns: 'track' (character varying (255)) and 'comments' (bigint). The table contains 17 rows of data, sorted by the number of comments in descending order. The first row is 'Dynamite' with 16083138 comments, and the last row is 'LOVE STOP ME' with 2312000 comments. The total number of rows is 14059, and the query completed in 00:00:00.161 seconds.

	track	comments
1	Dynamite	16083138
2	Butter	9131761
3	Boy With Luv (feat. Halsey)	6535721
4	Boy With Luv (feat. Halsey)	6535719
5	How You Like That	5130725
6	Baby	4805805
7	Despacito	4252791
8	Despacito	4252791
9	Pink Venom	3637659
10	Permission to Dance	3486944
11	DDU-DU DDU-DU	3312226
12	MIC Drop (Steve Aoki Remix) - Full Length Ed...	3109122
13	Yet To Come	2984558
14	Back Door	2782480
15	Drag Me Down	2469841
16	Kill This Love	2466445
17	LOVE STOP ME	2312000

Total rows: 14059 Query complete 00:00:00.161

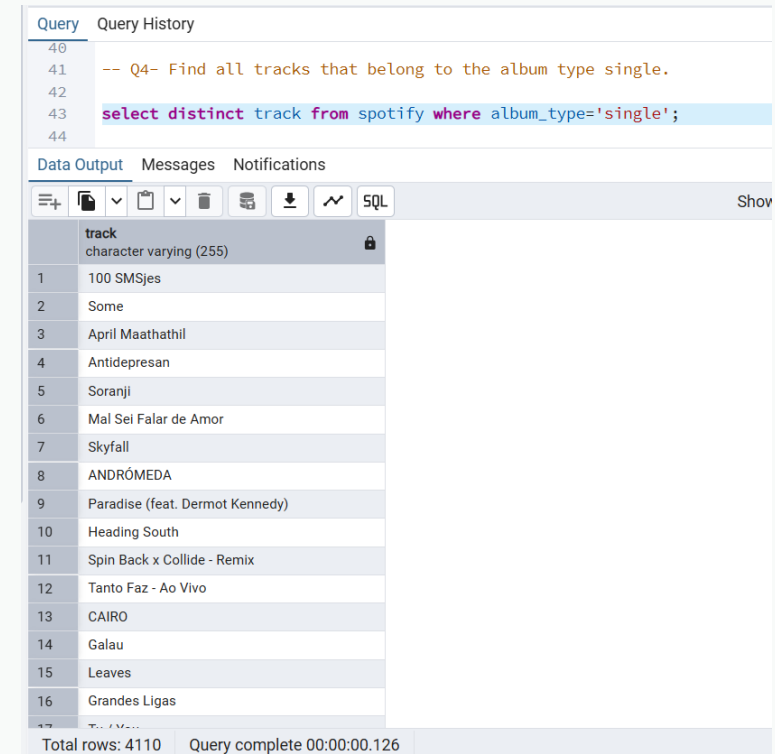
INTERPRETATION:

The result focuses the analysis on licensed content and shows comment engagement for 14,059 records.

Q4 — SQL Analysis

Find all tracks that belong to the album type 'single'.

```
SELECT DISTINCT track
FROM spotify
WHERE album_type = 'single';
```



Query Query History

```
40
41 -- Q4- Find all tracks that belong to the album type single.
42
43 select distinct track from spotify where album_type='single';
44
```

Data Output Messages Notifications

	track
	character varying (255)
1	100 SMSjes
2	Some
3	April Maathathil
4	Antidepressan
5	Soranjil
6	Mal Sei Falar de Amor
7	Skyfall
8	ANDRÔMEDA
9	Paradise (feat. Dermot Kennedy)
10	Heading South
11	Spin Back x Collide - Remix
12	Tanto Faz - Ao Vivo
13	CAIRO
14	Galau
15	Leaves
16	Grandes Ligas

Total rows: 4110 Query complete 00:00:00.126

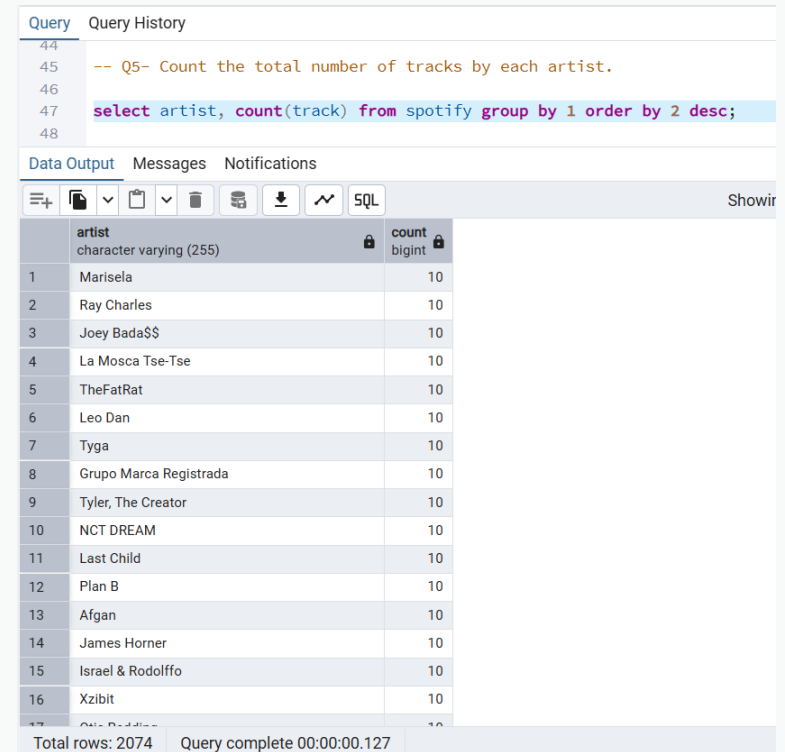
INTERPRETATION:

The filter returns 4,110 distinct tracks associated with the single category.

Q5 — SQL Analysis

Count the total number of tracks by each artist.

```
SELECT artist, COUNT(track)
FROM spotify
GROUP BY artist
ORDER BY 2 DESC;
```



The screenshot shows a SQL query editor with a query history tab. The query is: `-- Q5- Count the total number of tracks by each artist.`
`select artist, count(track) from spotify group by 1 order by 2 desc;`

Below the query editor is a table with the following data:

	artist character varying (255)	count bigint
1	Marisela	10
2	Ray Charles	10
3	Joey Bada\$\$	10
4	La Mosca Tse-Tse	10
5	TheFatRat	10
6	Leo Dan	10
7	Tyga	10
8	Grupo Marca Registrada	10
9	Tyler, The Creator	10
10	NCT DREAM	10
11	Last Child	10
12	Plan B	10
13	Afgan	10
14	James Horner	10
15	Israel & Rodolfo	10
16	Xzibit	10
17	Chloe Riddie	10

At the bottom of the table, it says: Total rows: 2074 Query complete 00:00:00.127

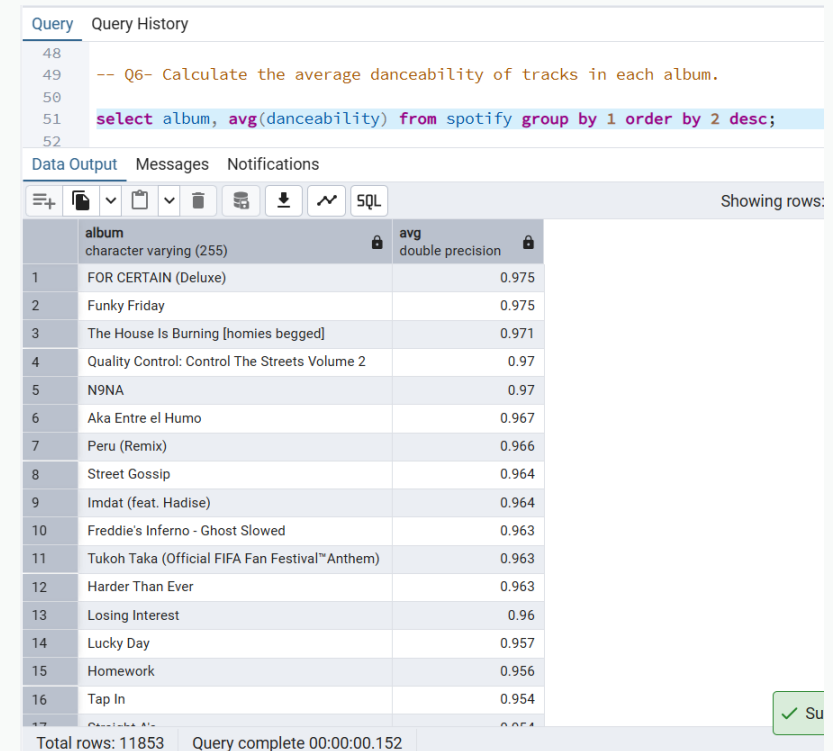
INTERPRETATION:

The query creates an artist-level track count. In the displayed result, 2,074 artists are represented.

Q6 — SQL Analysis

Calculate the average danceability of tracks in each album.

```
SELECT album, AVG(danceability)
FROM spotify
GROUP BY album
ORDER BY 2 DESC;
```



The screenshot shows a SQL query editor with a query history tab. The query is: `-- Q6- Calculate the average danceability of tracks in each album. select album, avg(danceability) from spotify group by 1 order by 2 desc;`. Below the query is a 'Data Output' tab showing a table with two columns: 'album' (character varying (255)) and 'avg' (double precision). The table contains 16 rows of data, sorted by average danceability in descending order. The first row is 'FOR CERTAIN (Deluxe)' with an average of 0.975, and the last row is 'Tap In' with an average of 0.954. The interface includes a toolbar with icons for query execution, saving, and other database functions. A status bar at the bottom indicates 'Total rows: 11853' and 'Query complete 00:00:00.152'.

	album character varying (255)	avg double precision
1	FOR CERTAIN (Deluxe)	0.975
2	Funky Friday	0.975
3	The House Is Burning [homies begged]	0.971
4	Quality Control: Control The Streets Volume 2	0.97
5	N9NA	0.97
6	Aka Entre el Humo	0.967
7	Peru (Remix)	0.966
8	Street Gossip	0.964
9	Imdat (feat. Hadise)	0.964
10	Freddie's Inferno - Ghost Slowed	0.963
11	Tukoh Taka (Official FIFA Fan Festival™ Anthem)	0.963
12	Harder Than Ever	0.963
13	Losing Interest	0.96
14	Lucky Day	0.957
15	Homework	0.956
16	Tap In	0.954

INTERPRETATION:

AVG() summarizes audio characteristics at album level and enables comparison of how danceable different albums are.

Q7 — SQL Analysis

Find the top 5 tracks with the highest energy values.

```
SELECT track, MAX(energy)
FROM spotify
GROUP BY track
ORDER BY 2 DESC
LIMIT 5;
```

Query

Query History

52

53

54

55

56

-- Q7- Find the top 5 tracks with the highest energy values.

select track, max(energy) from spotify group by 1 order by 2 desc limit 5;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing

	track character varying (255)	max double precision
1	Rain and Thunderstorm, Pt. 7	1
2	Rain and Thunderstorm, Pt. 33	1
3	Rain and Thunderstorm, Pt. 4	1
4	Rain and Thunderstorm, Pt. 6	1
5	Gentle Piano Melodies	1

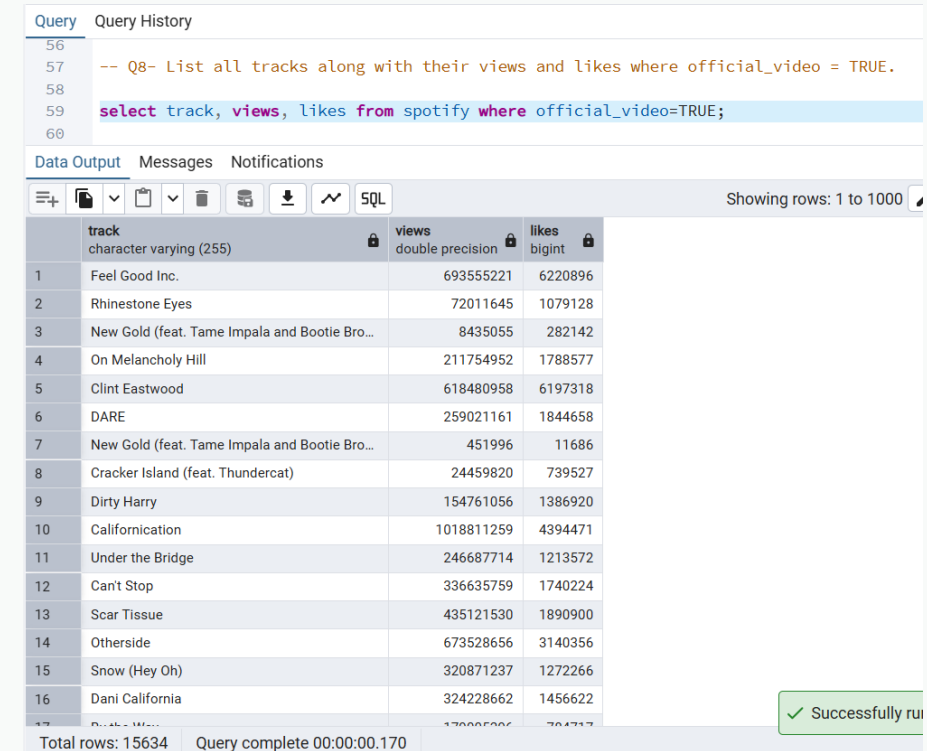
INTERPRETATION:

The top five returned tracks reach an energy value of 1.0, the maximum possible value in this dataset.

Q8 — SQL Analysis

List tracks with their views and likes where official_video = TRUE.

```
SELECT track, views, likes
FROM spotify
WHERE official_video = TRUE;
```



The screenshot shows a SQL query editor with a query history tab. The query is: `-- Q8- List all tracks along with their views and likes where official_video = TRUE.` followed by `select track, views, likes from spotify where official_video=TRUE;`. Below the query, there is a 'Data Output' tab showing a table with 16 rows. The table has columns: track (character varying (255)), views (double precision), and likes (bigint). The rows list various tracks and their corresponding views and likes. A status bar at the bottom indicates 'Total rows: 15634' and 'Query complete 00:00:00.170'. A green checkmark icon and the text 'Successfully run' are visible in the bottom right corner.

	track character varying (255)	views double precision	likes bigint
1	Feel Good Inc.	693555221	6220896
2	Rhinestone Eyes	72011645	1079128
3	New Gold (feat. Tame Impala and Bootie Bro...	8435055	282142
4	On Melancholy Hill	211754952	1788577
5	Clint Eastwood	618480958	6197318
6	DARE	259021161	1844658
7	New Gold (feat. Tame Impala and Bootie Bro...	451996	11686
8	Cracker Island (feat. Thundercat)	24459820	739527
9	Dirty Harry	154761056	1386920
10	Californication	1018811259	4394471
11	Under the Bridge	246687714	1213572
12	Can't Stop	336635759	1740224
13	Scar Tissue	435121530	1890900
14	Otherside	673528656	3140356
15	Snow (Hey Oh)	320871237	1272266
16	Dani California	324228662	1456622

Total rows: 15634 Query complete 00:00:00.170 ✓ Successfully run

INTERPRETATION:

This isolates official-video content and provides two engagement metrics—views and likes—for comparison.

Q9 — SQL Analysis

For each album, calculate the total views of all associated tracks.

```
SELECT album, SUM(views) AS total_views
FROM spotify
GROUP BY album
ORDER BY total_views DESC;
```

Album	Total Views
VIDA	21,140,883,569
See You Again (feat. Charlie Puth)	11,547,595,554
The Heist	11,240,587,663
Peace Is The Mission (Extended)	10,634,457,280
CoComelon Kids Hits, Vol. 1	10,518,197,319
÷ (Deluxe)	9,666,639,016
PRISM	7,820,546,400
What Is Love? (Deluxe Edition)	7,378,170,568

INTERPRETATION:

This is the corrected album-level aggregation: one row per album with the total views accumulated by its tracks. The earlier screenshot used an incorrect GROUP BY.

Q10 — SQL Analysis

Retrieve tracks streamed on Spotify more than on YouTube.

```
SELECT *
FROM (
    SELECT track,
           COALESCE(SUM(CASE WHEN most_played_on='Youtube' THEN
stream END),0) AS streamed_on_youtube,
           COALESCE(SUM(CASE WHEN most_played_on='Spotify' THEN
stream END),0) AS streamed_on_spotify
    FROM spotify
    GROUP BY track
) t1
WHERE streamed_on_spotify > streamed_on_youtube
AND streamed_on_youtube <> 0;
```

The screenshot shows a SQL query execution interface. At the top, there's a 'Query' tab with the following SQL code:

```
-- Q10- Retrieve the track names that have been streamed on Spotify more than YouTube.

select * from
(select track,
coalesce(sum(case when most_played_on='Youtube' then stream end),0) as streamed_on_youtube,
coalesce(sum(case when most_played_on='Spotify' then stream end),0) as streamed_on_spotify
from spotify group by 1) as t1
where streamed_on_spotify > streamed_on_youtube and streamed_on_youtube <> 0
```

Below the query is the 'Data Output' tab, which displays a table with 3 columns: 'track', 'streamed_on_youtube', and 'streamed_on_spotify'. The table shows 13 rows of data, with the first row being 'Usted' and the last row being 'No Diggity'. The 'streamed_on_youtube' column is numeric, and the 'streamed_on_spotify' column is numeric. The table is sorted by 'track'.

At the bottom of the interface, there's a status bar that says 'Total rows: 155' and 'Query complete 00:00:00.135'. A green box on the right side of the status bar says 'Successfully run. Total'.

	track character varying (255)	streamed_on_youtube numeric	streamed_on_spotify numeric
1	Usted	30059201	137916795
2	3 A.M.	31174412	607872704
3	21 Hungarian Dances, WoO 1: Hungarian Dan...	39575743	79151486
4	Mientes Tan Bien	6915455	224299945
5	Have You Ever Seen The Rain	61903001	975300588
6	Dream A Little Dream Of Me	157256901	495674374
7	When I Grow Up	231236307	260959663
8	Me Hace Tanto Bien	56694580	187498268
9	What You Want (feat. Total)	13099909	85458315
10	Suavemente	134326707	250550159
11	Emotions	218177	107654866
12	Do It Again	116968498	182238193
13	No Diggity	47145766	626845940

INTERPRETATION:

Conditional aggregation compares platform-specific stream totals. The screenshot returns 155 tracks meeting the stated condition.

Q11 — SQL Analysis

Find the top 3 most-viewed tracks for each artist using a window function.

```
WITH ranking_artist AS (  
  SELECT artist, track, SUM(views) AS total_views,  
         DENSE_RANK() OVER (  
           PARTITION BY artist ORDER BY SUM(views) DESC  
         ) AS rank  
  FROM spotify  
  GROUP BY artist, track  
)  
SELECT *  
FROM ranking_artist  
WHERE rank <= 3;
```

The screenshot shows a SQL query editor with a 'Query' tab. The query is: `-- Q11- Find the top 3 most-viewed tracks for each artist using window functions`
`with ranking_artist as (
 select artist, track, sum(views) as total_views,
 dense_rank() over(partition by artist order by sum(views) desc) as rank
 from spotify group by 1, 2 order by 1, 3 desc
)
select * from ranking_artist where rank <= 3`

Below the query is a 'Data Output' tab showing the results. The table has 4 columns: `artist` (character varying (255)), `track` (character varying (255)), `total_views` (double precision), and `rank` (bigint). The results show 13 rows, with the first 3 rows being the top 3 most-viewed tracks for each artist.

	artist character varying (255)	track character varying (255)	total_views double precision	rank bigint
1	\$NOT	Tell Em	41100657	1
2	\$NOT	Like Me (feat. iann dior)	15803517	2
3	\$NOT	Mean	13563870	3
4	\$uicideboy\$	Paris	175156959	1
5	\$uicideboy\$	For the Last Time	91771038	2
6	\$uicideboy\$	Kill Yourself (Part III)	31674988	3
7	(G)-I-DLE	POP/STARS	541864471	1
8	(G)-I-DLE	TOMBOY	204797335	2
9	(G)-I-DLE	Oh my god	191587399	3
10	*NSYNC	Bye Bye Bye	315874581	1
11	*NSYNC	This I Promise You	252377383	2
12	*NSYNC	It's Gonna Be Me	172368673	3
13	070 Shake	Escapism. - Slowed Down	25149399	

Total rows: 6808 Query complete 00:00:00.185

Showing rows: 1 to 100

Successful

INTERPRETATION:

DENSE_RANK() ranks tracks within each artist partition, making it possible to keep the top three tracks for every artist.

Q12 — SQL Analysis

Find tracks where the liveness score is above the overall average.

```
SELECT track, liveness
FROM spotify
WHERE liveness > (SELECT AVG(liveness) FROM spotify);
```

Query		Query History	
83	-- Q12- Write a query to find tracks where the liveness score is above the average.		
84			
85	select track, liveness from spotify where liveness > (select avg(liveness) from spotify)		
86			
Data Output		Messages	Notifications
Showing rows: 1 to 1000			
track	liveness		
character varying (255)	double precision		
1	Feel Good Inc.		0.613
2	DARE		0.298
3	New Gold (feat. Tame Impala and Bootie Bro...		0.325
4	Cracker Island (feat. Thundercat)		0.325
5	Dirty Harry		0.672
6	Dani California		0.346
7	Candy Shop		0.38
8	Just A Lil Bit		0.315
9	Disco Inferno		0.279
10	The Unforgiven (Remastered)		0.255
11	One (Remastered)		0.42
12	Screaming Suicide		0.239
13	Yellow		0.234
14	A Sky Full of Stars		0.209
15	Hymn for the Weekend		0.325
16	My Universe		0.328
17	One More Time		0.332
Total rows: 6364		Query complete 00:00:00.116	

INTERPRETATION:

A scalar subquery calculates the dataset-wide average, then the outer query returns tracks above that benchmark.

Q13 — SQL Analysis

Use a CTE to calculate the difference between the highest and lowest energy values in each album.

```
WITH cte AS (  
    SELECT album,  
           MAX(energy) AS max_energy,  
           MIN(energy) AS min_energy  
    FROM spotify  
    GROUP BY album  
)  
SELECT album,  
       max_energy - min_energy AS energy_diff  
FROM cte  
ORDER BY energy_diff DESC;
```

Query

Query History

87

88

89

90

91

92

93

/* Q13- Use a WITH clause to calculate the difference between the highest and lowest energy values for tracks in each album.*/

with cte as(
select album, max(energy) as max_energy, min(energy) as min_energy from spotify group by 1
)
select album, max_energy - min_energy as energy_diff from cte order by energy_diff desc

Data Output

Messages

Notifications

Showing rows: 1 to 1000

album

character varying (255)

energy_diff

double precision

1

White Noise

0.9067500000000001

2

Spotify Singles - Holiday

0.8360000000000001

3

Spotify Singles

0.8232

4

UNDERTALE Soundtrack

0.816

5

Making Mirrors

0.8109000000000001

6

Everytime We Touch (Premium Edition)

0.8049999999999999

7

If I Can Dream: Elvis Presley with the Royal Phi...

0.787

8

Fallen

0.7649999999999999

9

CeeLo's Magic Moment

0.762

10

Greatest Hits

0.741

11

Safe Haven

0.738

12

Pain Killer

0.734

13

Paranoid (Remaster)

0.7325

14

Nevermind (Remastered)

0.7110000000000001

Successfully run. Total

Total rows: 11853

Query complete 00:00:00.146

INTERPRETATION:

The CTE separates the aggregation step from the final calculation. Larger energy differences indicate greater variation within an album.

Q14 — SQL Analysis

Find tracks where the energy-to-liveness ratio is greater than 1.2.

```
SELECT track,  
       energy / NULLIF(liveness,0) AS e2l_ratio  
FROM spotify  
WHERE energy / NULLIF(liveness,0) > 1.2  
ORDER BY 2 DESC;
```

Query		Query History
95	-- Q14- Find tracks where the energy-to-liveness ratio is greater than 1.2	
96		
97	select track, (energy / nullif(liveness,0)) as e2l_ratio from spotify where	
98	(energy / nullif(liveness,0)) > 1.2	
99	order by 2 desc	
Data Output		Messages Notifications
Showing rows:		
	track	e2l_ratio
	character varying (255)	double precision
1	Take It	59.11392405063291
2	Verano Azul	58
3	Salvavidas	57.655172413793096
4	Ants Marching	54.52229299363058
5	Eres Mi Sueño - Versión Radio Edit	51.38121546961326
6	Feel My Needs - Purple Disco Machine Remix	49.230769230769226
7	Bonetrousle	48.9
8	Dominicana	45.527638190954775
9	Socadona (feat. Mr. Vegas)	43.77659574468085
10	Ain't It Fun	42.52380952380952
11	Mal Bicho - Remasterizado 2008	41.733333333333334
12	Psychosocial	40.370370370370374
13	CASE 143	40.09009009009009
14	Finesse - Remix; feat. Cardi B	39.95348837209303
15	Daydreamin'	39.64912280701754
16	Hollaback Girl	39.14529914529915
Total rows: 18797		Query complete 00:00:00.116

INTERPRETATION:

NULLIF() prevents division-by-zero errors. The resulting ratio highlights tracks with relatively high energy compared with liveness.

Q15 — SQL Analysis

Calculate the cumulative sum of likes for tracks ordered by views.

```
SELECT track, likes, views,  
       SUM(likes) OVER (ORDER BY views DESC) AS  
       cumulative_likes  
FROM spotify;
```

The screenshot shows a SQL query editor with a query history tab. The query is: `/* Q15- Calculate the cumulative sum of likes for tracks ordered by the number of views, using window functions.*/
select track, likes, views, sum(likes) over(order by views desc) as cumulative_likes from spotify`. Below the query is a table with 4 columns: track, likes, views, and cumulative_likes. The table contains 16 rows of data, sorted by views in descending order. The 'cumulative_likes' column shows the running total of likes for each track. At the bottom, there is a status bar indicating 'Total rows: 20592' and 'Query complete 00:00:00.138'. A green notification box on the right says 'Successfully run. Total query'.

	track character varying (255)	likes bigint	views double precision	cumulative_likes numeric
1	Despacito	50788652	8079649362	50788652
2	Despacito	50788626	8079646911	101577278
3	Shape of You	31047780	5908398479	132625058
4	See You Again (feat. Charlie Puth)	40147674	5773798407	172772732
5	See You Again (feat. Charlie Puth)	40147618	5773797147	212920350
6	Wheels on the Bus	14396841	4898831101	227317191
7	Uptown Funk (feat. Bruno Mars)	20067879	4821016218	247385070
8	Gangnam Style (?????)	26399133	4679767471	273784203
9	Sugar	15340646	3817733132	289124849
10	Roar	15864499	3725748519	304989348
11	Counting Stars	16558621	3721610307	321547969
12	Sorry	15789307	3627305503	337337276
13	Thinking out Loud	14343730	3547155867	351681006
14	Baa Baa Black Sheep	8943443	3486504328	360624449
15	Waka Waka (This Time for Africa) [The Offici...	20327883	3463816330	380952332
16	Dark Horse	16342399	3442526730	397294731

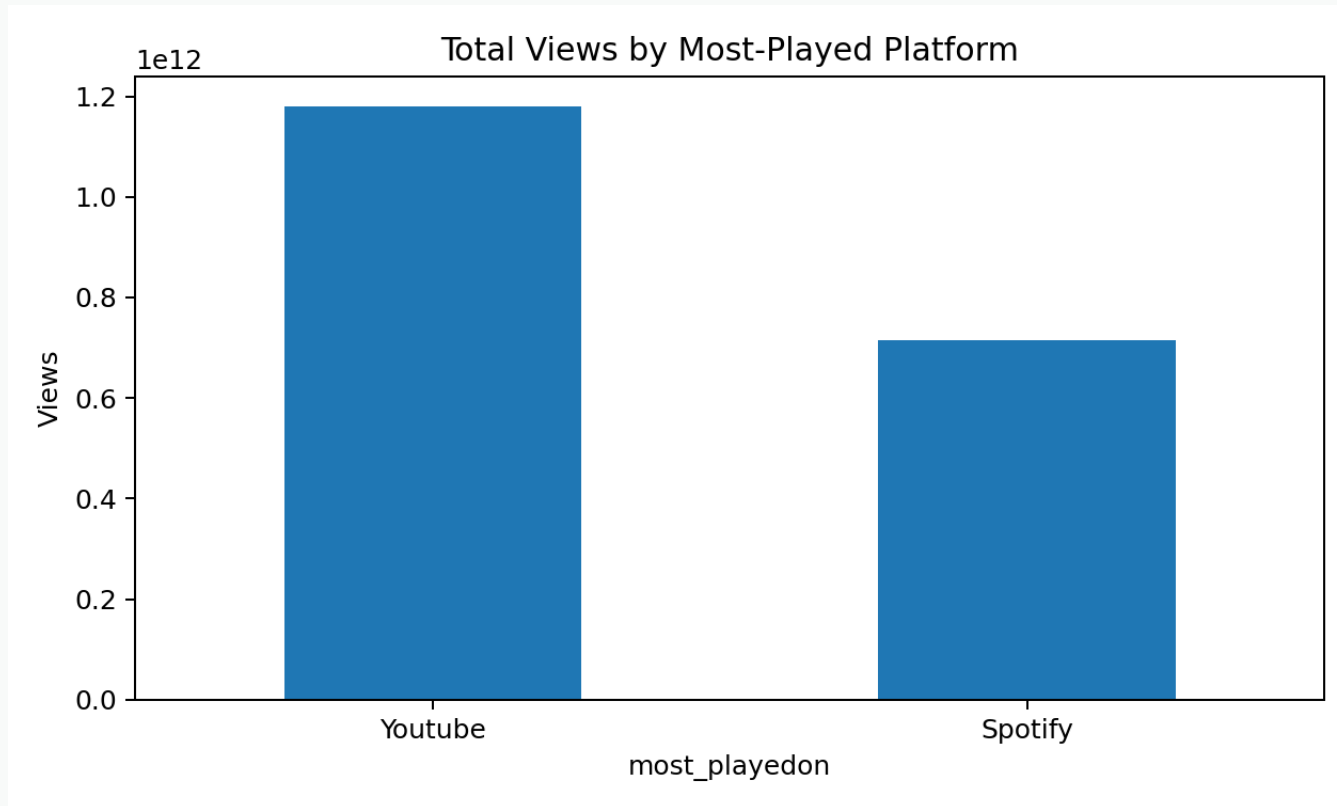
Total rows: 20592 Query complete 00:00:00.138 Successfully run. Total query

INTERPRETATION:

The windowed SUM() produces a running total of likes as tracks are traversed from highest to lower view counts.

Platform Engagement

Views aggregated by the most-played platform



- Platform-level aggregation is useful for understanding where content receives more exposure.
- The dataset contains both YouTube and Spotify as values in the `most_playedon` field.
- This complements Q10, which compares Spotify and YouTube stream counts at track level.

Data Analyst Skills Demonstrated

- Data exploration and validation before analysis.
- Writing SQL from business-style analytical questions.
- Aggregating data at artist, track and album levels.
- Comparing metrics and identifying top-performing content.
- Using advanced SQL for ranking, ratios and cumulative calculations.
- Handling edge cases with COALESCE() and NULLIF().
- Communicating results through concise interpretations and visual evidence.

Challenges & Learnings

- Duplicate track/album records require careful use of DISTINCT and appropriate grouping.
- Aggregation level matters: grouping by both album and track changes the meaning of a total.
- Boolean filters such as `licensed = TRUE` and `official_video = TRUE` are useful for segmenting content.
- Ratios can fail when a denominator is zero; `NULLIF()` provides a safe SQL pattern.
- Window functions are powerful when the analysis requires rankings within groups or running totals.

Recommendations & Analytical Extensions

- Build a Power BI/Tableau dashboard for artist, album and platform performance.
- Create KPIs for views, likes, comments, streams and engagement ratios.
- Investigate which audio characteristics are associated with higher views or streams.
- Segment tracks by album type, platform and official-video status.
- For future database-focused work, study indexes and EXPLAIN ANALYZE after core SQL analysis skills are comfortable.

Conclusion